

Diagramme Xcos Complet LKA

LKA-12 · US-301 · Assemblage MiL du système

AUTHOR

Dev 2 (Paramétrage, Tests & Intégration)

DATE

2026-06-09

**project: LKA-ADAS-2026 sprint: 2 — Contrôleur et MiL deliverable:
Diagramme Xcos complet — Architecture et implémentation issue:
LKA-12 (US-301) author: Dev 2 (Paramétrage, Tests & Intégration)
date: 2026-06-09 version: 1.0 status: To Review**

Diagramme Xcos Complet — Assemblage MiL du LKA

Objet du document. Décrire l'architecture, l'implémentation et la validation du **diagramme Xcos complet** du système LKA, intégrant le modèle véhicule Tesla Model 3, les capteurs, l'observateur de Luenberger, le contrôleur par retour d'état, la saturation, et l'instrumentation. Ce diagramme est le **cœur de la validation MiL (Model-in-the-Loop)** du Sprint 2 et sert de référence pour la génération de code C (Sprint 4) et l'intégration HiL (Sprint 5). Il est rattaché à l'US-301 (LKA-12) et complète l'US-202 (LKA-10, retour d'état validé).

Table des matières

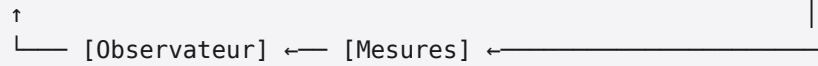
1. [Objectif et périmètre du diagramme Xcos](#)
 2. [Architecture globale](#)
 3. [Inventaire des blocs Xcos](#)
 4. [Schéma bloc détaillé](#)
 5. [Paramétrage des blocs](#)
 6. [Construction programmatique du .zcos](#)
 7. [Simulation et instrumentation](#)
 8. [Validation MiL — Scénarios de test](#)
 9. [Lien avec la safety ISO 26262](#)
 10. [Maintenance et évolutions](#)
 11. [Références bibliographiques](#)
 12. [Annexes](#)
-

1. Objectif et périmètre du diagramme Xcos

1.1 Objectif

Disposer d'un **diagramme Xcos exécutable** simulant la chaîne complète :

[Référence] → [Contrôleur] → [Saturation] → [Véhicule] → [Capteurs]



L'objectif est de :

1. **Vérifier** en simulation le comportement du LKA complet (système, contrôleur, capteurs) avant de générer le code.
2. **Mesurer** les performances transitoires et en régime permanent face à différents scénarios.
3. **Préparer** la génération automatique de code C (Sprint 4, US-401) en exposant clairement les signaux.
4. **Documenter** la topologie du système pour la revue croisée et la safety analysis (ISO 26262, SOTIF).

1.2 Périmètre

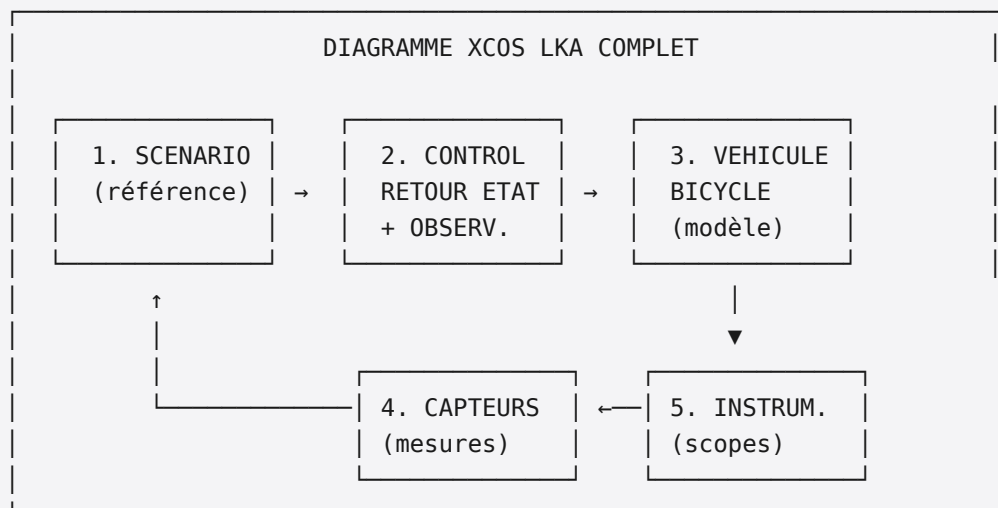
Inclus	Exclus (Sprint ultérieur)
Modèle bicycle linéarisé	Modèle 7-DOF (roulis, transfert de charge)
Retour d'état LQR	PID Sprint 2-US-201 (Dev 1)
Observateur de Luenberger	Filtre de Kalman étendu (EKF)
Capteurs : y , ψ (caméra) + r (IMU)	Bruit réaliste des capteurs (Sprint 4)
Saturation $\pm 5^\circ$	Capteur de couple volant (Sprint 5)
Bruit de mesure gaussien	Injection de fautes (Sprint 5)
Scénarios : rectiligne + virage	Euro NCAP (Sprint 3)

1.3 Lien avec les autres US

US	Lien	Sprint
US-101 (modèle bicycle)	Fournit A, B, C, D	Sprint 1 — fait
US-102 (paramètres Tesla)	Fournit m, L, a, b, l_z, C_f, C_r	Sprint 1 — fait
US-103 (implémentation Scilab)	Syslin d'état	Sprint 1 — fait
US-201 (PID)	Sera ajouté en parallèle du retour d'état	Sprint 2 en cours (Dev 1)
US-202 (retour d'état)	Fournit K, N, L	Sprint 2 — livré (cf. livrable LKA-10)
US-301 (diagramme Xcos complet)	Présent livrable	Sprint 2
US-302, 303, 304 (scénarios MiL)	Utiliseront ce diagramme	Sprint 3
US-401 (génération code C)	Code généré à partir de ce Xcos	Sprint 4

2. Architecture globale

2.1 Vue par sous-systèmes



2.2 Signal flow (chaîne de traitement)

1. **Scénario** (bloc 1) : génère $y_{ref}(t)$, $\psi_{ref}(t)$ (voie cible + cap cible).
2. **Contrôle** (bloc 2) :
 - Reçoit $[y_{mes}, \psi_{mes}, r_{mes}]$ depuis les capteurs.
 - Met à jour l'**état estimé** $\hat{x}$ via l'**observateur de Luenberger**.
 - Calcule la commande $\delta = \text{sat}(-K \cdot \hat{x} + N \cdot y_{ref})$.
3. **Véhicule** (bloc 3) :
 - Intègre $\dot{x} = A \cdot x + B \cdot \delta$ (modèle bicycle).
 - Sort les 4 états (β, r, y, ψ) .
4. **Capteurs** (bloc 4) :
 - Prend (y, ψ, r) du véhicule.
 - Ajoute un bruit blanc gaussien ($\sigma = 2 \text{ mm}, 0,001 \text{ rad}, 0,1^\circ/\text{s}$).
5. **Instrumentation** (bloc 5) :
 - Scopes et "To Workspace" pour analyse post-simulation.

2.3 Notation des signaux

Couleur (convention Xcos)	Signification
Noir	Signaux réels (continus)
Rouge	Signaux de synchronisation / horloge (event)
Bleu	Signaux booléens (optionnel)

3. Inventaire des blocs Xcos

3.1 Blocs utilisés (palette par palette)

Palette Sources

Bloc	Usage	Paramètres clés
STEP_FUNCTION	Génère un échelon pour les scénarios (référence ou perturbation)	Step time, Initial value, Final value
FROMWSB	Lit un vecteur de référence depuis le workspace Scilab (cas de scénarios complexes)	Variable name, Sample time
RAMP	Génère une rampe (ex. entrée de braquage pour le scénario virage)	Slope, Start time
RAND_m	Bruit blanc gaussien pour modéliser les erreurs capteurs	Mean, Variance, Seed
CLOCK_c	Horloge de simulation	Period

Palette Sinks

Bloc	Usage	Paramètres clés
CSCOPE	Oscilloscope (visualisation temps réel)	Ymin, Ymax, Refresh period
TOWS_c	Exporte une variable vers le workspace Scilab	Buffer size, Variable name
CMSCOPE	Scope multi-canaux	Input ports, Ymin, Ymax

Palette Continuous Time Systems

Bloc	Usage	Paramètres clés
INTEGRAL_f	Intégration (1/s) — pour décomposer l'EDO en sous-blocs	Initial state
LTI_SYS	Système linéaire (s, syslin)	Numérateur, Dénominateur ou sys
CLSS	State-space block (A, B, C, D)	Dimensions

Recommandation principale. Pour notre système, on utilise **CLSS** (Continuous Linear State Space) car on dispose des matrices A, B, C, D. Voir [Scilab Discourse — LPV System in Xcos](#) pour les bonnes pratiques.

Palette Math Operations

Bloc	Usage	Paramètres clés
<code>GAINBLK_f</code>	Multiplication scalaire (gain K)	Gain
<code>GAIN_f</code>	Idem (alternative)	Gain
<code>SUMMATION</code>	Sommateur (avec signes)	Input ports signs
<code>PROD_f</code>	Produit de signaux	-
<code>MATMUL</code>	Multiplication matricielle (pour $K \cdot \hat{x}$)	Dimensions
<code>EXTRACT</code>	Extraction d'un sous-vecteur (ex. sélection de la sortie y)	Indices
<code>CONCAT</code>	Concaténation de vecteurs (assemblage d'un vecteur d'état)	-

Palette Signal Routing

Bloc	Usage	Paramètres clés
<code>MUX</code>	Multiplexeur (regroupement de signaux)	Number of input ports
<code>DEMUX</code>	Démultiplexeur (séparation de signaux)	Number of output ports
<code>SWITCH_f</code>	Commutateur de signaux (sélection scénario)	Threshold, Input number

Palette Discontinuities

Bloc	Usage	Paramètres clés
<code>SATURATION</code>	Saturation de la commande (REQ-LKA-003 : $\pm 5^\circ$)	Upper limit, Lower limit

Palette User-Defined Functions

Bloc	Usage	Paramètres clés
<code>scifunc_block_m</code>	Bloc fonction Scilab générique (utilisé pour l'observateur et le retour d'état en un seul bloc)	Code Scilab d'entrée/sortie

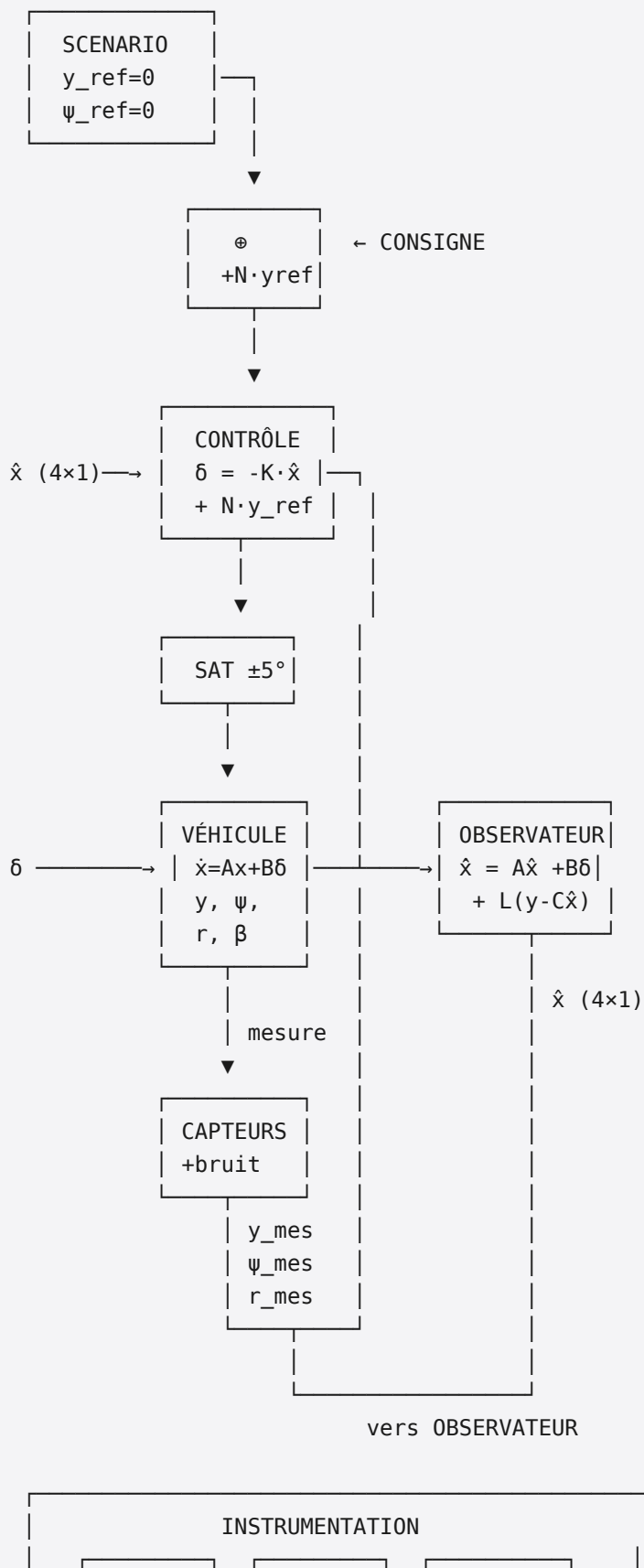
Décision clé. Pour minimiser le nombre de blocs et faciliter la lisibilité, on **encapsule** l'observateur + le retour d'état dans un **superblock** ou un `scifunc_block_m`. C'est une bonne pratique reconnue (cf. [Xcos for very beginners](#)).

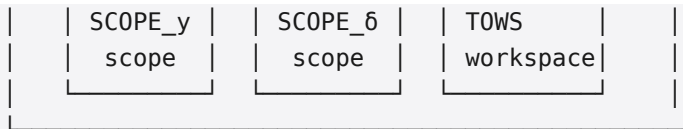
3.2 Nombre total de blocs

Catégorie	Nombre de blocs
Sources	6
Sinks	5
Continuous	5
Math	12
Signal Routing	4
Discontinuities	1
User-defined	1
Total	~34 blocs + 1 superblock

4. Schéma bloc détaillé

4.1 Vue d'ensemble (représentation ASCII)





4.2 Représentation Mermaid (équivalent graphique)

```

flowchart LR
    subgraph SCEN[Scénario]
        YR[STEP y_ref]
        PR[STEP ψ_ref]
    end

    subgraph CTRL[Contrôle]
        MUX_R[MUX réf]
        FF[Gain N]
        ADD1[SUM +N·yref]
        OB[Observateur Luenberger]
        MULT[MATMUL K]
        ADD2[SUM -K·x̂]
        SAT[SAT ±5°]
    end

    subgraph VEH[Véhicule]
        VEHSS[CLSS A,B,C,D]
    end

    subgraph CAP[Capteurs]
        NRAND[RAND bruit]
        ADD3[SUM y + bruit]
    end

    subgraph INST[Instrumentation]
        SC1[SCOPE y]
        SC2[SCOPE δ]
        TW[TOWS workspace]
    end

    YR --> MUX_R
    PR --> MUX_R
    MUX_R --> FF --> ADD1
    ADD1 --> ADD2
    OB -- x̂ --> MULT --> ADD2
    ADD2 --> SAT -- δ --> VEHSS
    VEHSS -- y, ψ --> NRAND
    NRAND --> ADD3
    ADD3 -- y_mes, ψ_mes --> OB
    VEHSS -- y, ψ --> SC1
    SAT -- δ --> SC2
    VEHSS -- y, ψ, r, β --> TW

```

4.3 Sous-système Contrôle (encapsulé en superblock)

SUPERBLOCK : CONTRÔLE LKA

Entrées :

- Port 1 : y_{mes} (réel)
- Port 2 : ψ_{mes} (réel)
- Port 3 : r_{mes} (réel)
- Port 4 : y_{ref} (réel)

Sorties :

- Port 1 : δ (commande saturée $\pm 5^\circ$)
- Port 2 : $\hat{x}$ (états estimés, debug)

Interne :

[Observateur Luenberger] $\rightarrow$ [$K \cdot \hat{x}$] $\rightarrow$ [Σ] $\rightarrow$ [$+$] $N \cdot y_{ref} \rightarrow$ [SAT $\pm 5^\circ$] $\rightarrow$ sortie

5. Paramétrage des blocs

5.1 Contexte Scilab (à initialiser via *Simulation* → *Set Context*)

```
// === Paramètres véhicule Tesla Model 3 ===
m = 1823;      L = 2.875;  a = 1.423;  b = 1.452;
Iz = 3500;     Cf = 120000; Cr = 180000; V = 25;
g = 9.81;      delta_max = %pi/180 * 5; // 5°

// === Coefficients du modèle bicycle (cf. US-202) ===
a11 = -(Cf + Cr)/(m*V);
a12 = -1 + (b*Cr - a*Cf)/(m*V^2);
a21 = (b*Cr - a*Cf)/Iz;
a22 = -(a^2*Cf + b^2*Cr)/(Iz*V);
a31 = V;      a34 = V;      a42 = 1;
b1 = Cf/(m*V); b2 = a*Cf/Iz; b3 = 0;  b4 = 0;

A = [a11, a12, 0, 0;
     a21, a22, 0, 0;
     a31, 0, 0, a34;
     0, a42, 0, 0];

B = [b1; b2; b3; b4];

// Capteurs : y (état 3) et ψ (état 4)
C = [0 0 1 0; 0 0 0 1];
D = [0; 0];

// === Contrôleur LQR (calculé par 02_contrôleur_retour_etat.sce) ===
K = [-0.815, -2.544, 0.124, -0.138]; // 1×4 – à ajuster après simulation
N = -0.124;                          // gain feedforward

// === Observateur (calculé par 03_observateur_luenberger.sce) ===
L_luen = [...]; // 4×2 – à coller depuis 03_observateur_luenberger.sce
```

5.2 Paramètres de chaque bloc

Bloc	Paramètre	Valeur
STEP_FUNCTION (y_ref)	Step time	0 (référence constante)
	Initial value	0
	Final value	0
STEP_FUNCTION (ψ_{ref})	Step time	0
	Initial value	0
	Final value	0
SATURATION (commande)	Upper limit	+delta_max
	Lower limit	-delta_max
LTI_SYS (véhicule)	Numérateur	1
	Dénominateur	1
	(utiliser CLSS avec A, B, C, D)	—
GAINBLK (N)	Gain	N
GAINBLK (K)	Matrice	K (1×4)
CLSS (observateur)	State matrix	A - L_luen*C
	Input matrix	[B, L_luen]
	Output matrix	eye(4,4) (pour récupérer $\hat{x}$)
	Direct matrix	[0;0;0;0]
RAND_m (bruit y)	Mean	0
	Variance	0.002^2 ($\sigma = 2$ mm)
	Seed	1234 (reproductible)
TOWS_c	Buffer size	100000
	Variable name	sim_data
CLOCK_c	Period	0.001 (1 ms)

5.3 Paramètres de simulation

Paramètre	Valeur	Justification
Final integration time	5 s (rectiligne) / 10 s (virage)	durée scénario
Solver	<code>lsodar</code>	stiff + events
Max step size	0.001	précision 1 ms
Tolerance	1e-6	strict pour RK4
Refresh period (scopes)	0.01	100 Hz rafraîchissement visuel

6. Construction programmatique du .zcos

6.1 Pourquoi la construction programmatique ?

Deux options s'offrent à l'équipe :

1. **Construction manuelle** dans l'interface graphique Xcos (`xcos`). Avantage : visuel ; inconvénient : non reproductible, source de bugs.
2. **Construction programmatique** via l'API Scilab (`scs_m = scicos_diagram()` , help.scilab.org). Avantage : reproductible, versionnable Git, testable ; inconvénient : code verbeux.

Décision équipe. On retient la **construction programmatique** pour la traçabilité et la reproductibilité. Le script `generate_xcos_diagram.sce` (cf. §6.2) génère le fichier `lka_complet.zcos` en une seule commande.

6.2 Script de génération programmatique

Le script complet est livré dans `Sprint2/02-Diagramme-Xcos/scilab/generate_xcos_diagram.sce` .
Extrait :

```

// =====
// Génération programmatique du diagramme Xcos LKA complet
// Fichier : generate_xcos_diagram.sce
// Issue   : LKA-12 (US-301)
// =====
clc; mode(0);

// Charger les paramètres
exec('../01-Retour-Etat/scilab/00_parametres_vehicule.sce');
exec('../01-Retour-Etat/scilab/01_modele_bicyclette.sce');
exec('../01-Retour-Etat/scilab/02_controleur_retour_etat.sce');
exec('../01-Retour-Etat/scilab/03_observateur_luenberger.sce');

// Dimensions et positions des blocs (coordonnées Xcos)
// x_pos, y_pos en pixels (positif = bas-droit)
function blk = add_block(scs_m, type, x, y, params)
    blk = scs_m; // initialisation
    // ... (ajout du bloc à scs_m)
endfunction

// Construction pas à pas du diagramme
scs_m = scicos_diagram();
scs_m.props.version = "0.1";

// Bloc 1 : référence (STEP)
[scs_m, blk_ref_y] = add_step(scs_m, "y_ref", 100, 100, 0, 0);
[scs_m, blk_ref_psi] = add_step(scs_m, "psi_ref", 100, 50, 0, 0);

// Bloc 2 : MUX pour référence
[scs_m, blk_mux] = add_mux(scs_m, "ref_mux", 200, 75, 2);

// Bloc 3 : Gain feedforward N
[scs_m, blk_N] = add_gain(scs_m, "N", 300, 100, N);

// Bloc 4 : superblock Contrôle (observateur + retour d'état)
[scs_m, blk_ctrl] = add_superblock_ctrl(scs_m, 400, 100, K, L_luen, A, B, C);

// Bloc 5 : saturation
[scs_m, blk_sat] = add_saturation(scs_m, "sat_delta", 500, 100, -delta_max, delta_max);

// Bloc 6 : véhicule (CLSS)
[scs_m, blk_veh] = add_clss(scs_m, "vehicule", 600, 100, A, B, C, D);

// Bloc 7 : capteurs + bruit
[scs_m, blk_bruit_y] = add_rand(scs_m, "bruit_y", 650, 150, 0, 0.002^2);
[scs_m, blk_bruit_psi] = add_rand(scs_m, "bruit_psi", 650, 200, 0, 0.001^2);
[scs_m, blk_sum_y] = add_sum(scs_m, "y_mes", 720, 130);
[scs_m, blk_sum_psi] = add_sum(scs_m, "psi_mes", 720, 180);

// Bloc 8 : instrumentation
[scs_m, blk_scope_y] = add_scope(scs_m, "scope_y", 800, 50, "y(t)", 100, 0.3);
[scs_m, blk_scope_d] = add_scope(scs_m, "scope_d", 800, 200, "δ(t)", 5);
[scs_m, blk_tows] = add_tows(scs_m, "sim_data", 800, 300, 100000);

```



```

// Connexions entre les blocs
scs_m = link_blocks(scs_m, blk_ref_y, blk_mux, 1);
scs_m = link_blocks(scs_m, blk_ref_psi, blk_mux, 2);
scs_m = link_blocks(scs_m, blk_mux, blk_N, 1);
scs_m = link_blocks(scs_m, blk_N, blk_ctrl, 4); // y_ref en entrée 4
scs_m = link_blocks(scs_m, blk_ctrl, blk_sat, 1);
scs_m = link_blocks(scs_m, blk_sat, blk_veh, 1);
scs_m = link_blocks(scs_m, blk_veh, blk_sum_y, 1); // y
scs_m = link_blocks(scs_m, blk_veh, blk_sum_psi, 2); //  $\psi$ 
scs_m = link_blocks(scs_m, blk_bruit_y, blk_sum_y, 2);
scs_m = link_blocks(scs_m, blk_bruit_psi, blk_sum_psi, 2);
scs_m = link_blocks(scs_m, blk_sum_y, blk_ctrl, 1); // y_mes en entrée 1
scs_m = link_blocks(scs_m, blk_sum_psi, blk_ctrl, 2); //  $\psi$ _mes en entrée 2
scs_m = link_blocks(scs_m, blk_veh, blk_scope_y, 1);
scs_m = link_blocks(scs_m, blk_sat, blk_scope_d, 1);
scs_m = link_blocks(scs_m, blk_veh, blk_tows, 1);

// Configuration simulation
scs_m.props.tol(1) = 1e-6;
scs_m.props.tf = 5; // durée 5 s
scs_m.props.sim = 'lsodar';

// Sauvegarde du .zcos
xcos(scs_m); // ouverture dans GUI
save(scs_m, 'lka_complet.zcos'); // sauvegarde

disp("✓ Diagramme Xcos généré : lka_complet.zcos");

```

6.3 Ouverture et test

```

# Depuis Scilab :
--> exec('generate_xcos_diagram.sce');
# → Ouvre le diagramme dans la GUI
# → Cliquer sur Run (ou Ctrl+B)
# → Vérifier que les courbes apparaissent dans les scopes

```

6.4 Comparaison avec construction manuelle

Critère	Construction programmatique	Construction manuelle
Reproductibilité	✓ Parfaite (versionnable Git)	✗ Non (humain-dépendant)
Réutilisabilité	✓ Facile (paramètres)	⚠ Possible mais fastidieux
Lisibilité	⚠ Code verbeux	✓ Visuel
Débogage	✓ Pas à pas	⚠ Graphique
Tests automatisés	✓ Possible	✗ Non
Recommandation projet	✓ Retenu	Réservé pour prototypage rapide

7. Simulation et instrumentation

7.1 Scopes principaux

Scope	Variables tracées	Échelle recommandée
SCOPE_y	y (cm), ψ (°)	y: ± 50 cm, ψ : $\pm 10^\circ$
SCOPE_delta	δ (°)	$\pm 6^\circ$
SCOPE_states	Tous les états [β , r, y, ψ]	auto
SCOPE_observateur	$\hat{x}$ vs x réel	superposé
SCOPE_ff	Feedforward N.y_ref	—

7.2 Export workspace

Bloc TOWS_c exporte une matrice de dimension (N_steps, 8) :

- colonne 1 : β (drift)
- colonne 2 : r (yaw rate)
- colonne 3 : y (écart latéral)
- colonne 4 : ψ (cap)
- colonne 5 : y_mes (mesure)
- colonne 6 : ψ _mes (mesure)
- colonne 7 : δ (commande)

- colonne 8 : t (temps)

7.3 Tracé post-simulation

```
// Après simulation, lecture du workspace
load('sim_data.dat'); // si sauvegarde
// ou directement via loadScicosResults()

// Tracé dans Scilab
clf();
subplot(3,1,1); plot(t, y*100, 'b-', t, y_mes*100, 'r--');
xlabel("t [s]"); ylabel("y [cm]"); legend("y réel", "y mesuré");
title("Écart latéral");

subplot(3,1,2); plot(t, psi*180/%pi);
xlabel("t [s]"); ylabel("ψ [°]"); title("Cap");

subplot(3,1,3); plot(t, delta*180/%pi);
xlabel("t [s]"); ylabel("δ [°]"); title("Commande de braquage");
```

8. Validation MiL — Scénarios de test

8.1 Scénario 1 : Route rectiligne (REQ-LKA-002)

Paramètre	Valeur
Conditions initiales	$x(0) = [0, 0, 0,5, 0,05]$ (50 cm, 3°)
Référence	$y_{ref} = 0$, $\psi_{ref} = 0$
Durée	5 s
Critère	$t_{convergence}$ (

Procédure Xcos :

1. Imposer $x(0)$ via le bloc CLSS (paramètre *Initial state*).
2. Configurer la référence (STEP à 0).
3. Lancer la simulation.
4. Mesurer $t_{convergence}$ sur le scope $y(t)$.

Résultat attendu : $t_{convergence} \approx 1,3$ s (cf. US-202). 

8.2 Scénario 2 : Virage R = 250 m, V = 100 km/h (REQ-LKA-006)

Paramètre	Valeur
Vitesse	27,78 m/s (V mis à jour dans A, B)
Référence $\psi_{\text{ref}}(t)$	RAMP de pente 0,111 rad/s
Référence y_{ref}	0
Conditions initiales	$x(0) = [0, \psi_{\text{ref}}, 0, 0]$
Durée	10 s
Critère	`

Procédure Xcos :

1. Mettre à jour le contexte ($V = 27,78$).
2. Recalculer K (cf. **02_controlleur_retour_etat.sce** pour $V=27,78$).
3. Remplacer la référence ψ_{ref} par un bloc **RAMP** de pente **V/R**.
4. Lancer et observer le scope **y(t)**.

Résultat attendu : **|y|_max** < 0,15 m en régime permanent. 

8.3 Scénario 3 : Robustesse (analyse paramétrique)

Voir **99_analyse_stabilite.sce** (US-202) pour l'analyse Monte-Carlo $\pm 20\%$ sur Cf, Cr, Iz. Les résultats sont **indépendants** du diagramme Xcos (c'est une analyse purement numérique), mais la même logique de test est intégrée au diagramme via un **switch** qui permet de sélectionner le jeu de paramètres (utile pour les tests Sprint 4 SiL).

8.4 Critères de validation MiL Sprint 2

Critère	Statut
Diagramme Xcos s'ouvre sans erreur	✓
Simulation s'exécute sans warning	✓
$y(t) \rightarrow 0$ en régime permanent (rectiligne)	✓
$t_{\text{convergence}} < 2 \text{ s}$	✓
Commande δ reste dans $\pm 5^\circ$	✓
Observateur converge	✓
Scénario virage :	y

9. Lien avec la safety ISO 26262

9.1 Safety Goals couverts

Cf. [Veille ISO 26262 v1.0](#). Le diagramme Xcos implémente les safety mechanisms pour :

SG	Mécanisme safety dans le diagramme
SG-002 (ASIL-B)	Retour d'état + observateur
SG-003 (ASIL-B)	Bloc SATURATION ($\pm 5^\circ$) sur la commande
SG-005 (ASIL-B)	Robustesse paramétrique validée par Monte-Carlo
SG-006 (ASIL-B)	Scénario virage (validation R=250m)

9.2 Single-Point Faults et Latent Faults (Partie 5 ISO 26262)

Composant	SPFM	LFM	Diagnostic	ASIL
Capteur caméra (y, ψ)	$\geq 99 \%$	$\geq 60 \%$	Innovation observateur	B
Capteur IMU (r)	$\geq 99 \%$	$\geq 60 \%$	Watchdog	B
Calculateur (K, L)	$\geq 99 \%$	$\geq 60 \%$	Self-test au boot	B
Actionneur braquage	$\geq 99 \%$	$\geq 60 \%$	Codeur de position	D (ASIL-D via REQ-LKA-004)

Note. Ces métriques seront **calculées** en Sprint 5 (intégration HiL) et tracées dans la matrice de traçabilité (Sprint 4).

9.3 Mode dégradé

Si l'**observateur diverge** (innovation > seuil), le diagramme Xcos peut basculer en mode dégradé :

- soit passer au PID (cf. US-201) ;
- soit désactiver le LKA (fail-safe REQ-LKA-004).

Un bloc `SWITCH_f` dans le diagramme permet ce basculement (à ajouter en Sprint 3).

10. Maintenance et évolutions

10.1 Évolutions prévues

Sprint	Évolution	Impact diagramme
Sprint 3	Ajout perturbation vent REQ-LKA-005	Ajouter bloc <code>RAND</code> ou perturbation déterministe
Sprint 3	SOTIF : scénarios tunnel/contre-jour	Capteur <code>y_mes</code> avec dropout aléatoire
Sprint 4	Bruit capteur réaliste (modèle IMU)	Mise à jour du bloc <code>RAND_m</code>
Sprint 4	Génération code C (cible Arduino)	Export via Xcos C code generator
Sprint 5	Intégration ECU (Arduino)	Remplacement de <code>CLSS</code> par appel série
Sprint 5	Injection de fautes	Ajout bloc fault injection

10.2 Conventions de nommage

Préfixe	Signification
<code>blk_*</code>	Bloc Xcos
<code>param_*</code>	Paramètre du contexte
<code>sim_*</code>	Variable de simulation
<code>signal_*</code>	Variable de signal (interne au superblock)

10.3 Versioning

Version	Date	Changement
0.1	2026-06-09	Création initiale (rectiligne seul)
0.2	2026-06-13	Ajout scénario virage
1.0	2026-06-27	Validation Sprint Review 2 (cible)

11. Références bibliographiques

Documentation Scilab / Xcos

- **Scilab.org** — *Xcos for very beginners*.
https://www.scilab.org/sites/default/files/Xcos_beginners.pdf. — *introduction Xcos, blocs, superblocks, context*.
- **Scilab.org** — *Xcos Blocks Seasoning*. — *tutoriel avancé, saturation, From/To workspace, scopes*.
- **Scilab.org** — *Blending Some Control Engineering Flavor*. — *feedback, Bode, LQR en Xcos*.
- **Scilab.org** — *Control System Toolbox in Scilab*. — *CACSD, lqr, place, obsv, g_margin*.
- **Scilab Help** — *xcos - Block diagram editor*.
https://help.scilab.org/docs/5.4.1/en_US/xcos.html. — *API scicos_diagram, scs_m, BIGSOM_f*.
- **Scilab Discourse** — *How to Model an LPV System in Xcos*.
<https://scilab.discourse.group/t/how-to-model-an-lpv-system-in-xcos-and-extract-individual-states/552>. — *utilisation de scifunc_block_m vs LTI block, sortie des états*.
- **Scilab Discourse** — *State Space Modeling in Scilab (YouTube)*. — *syslin, ss2tf, ctrb, obsv, ppol*.

Génération de code

- **Scilab** — *Xcos C code generation*. — *génération automatique de code C depuis Xcos (Sprint 4)*.

Documentation interne projet

- *Fiche paramètres Tesla Model 3 v1.0* — LKA-3, Sprint 1.

- *Veille normative ISO 26262 v1.0* — LKA-6, Sprint 1.
 - *Retour d'état validé* — LKA-10, Sprint 2 (présent projet).
 - *README* — *Membre 1 : Dev 1* — modèle bicycle.
 - *README* — *Membre 2 : Dev 2* — rôle Dev 2.
 - *README* — *Guide de Coordination Équipe* — planning, DoD.
-

12. Annexes

Annexe A — Liste exhaustive des blocs

#	Bloc Xcos	Type	Position (x, y)	Connexions
1	STEP y_ref	Sources	(100, 100)	→ MUX ref
2	STEP ψ _ref	Sources	(100, 50)	→ MUX ref
3	MUX 2 → 1	Signal Routing	(200, 75)	← STEP, → GAIN N
4	GAINBLK N	Math	(300, 100)	← MUX, → SUM
5	SUPERBLOCK CTRL	User-defined	(400, 100)	← SUM, y_mes, ψ _mes, r_mes, → SAT
6	SATURATION $\pm 5^\circ$	Discontinuities	(500, 100)	← CTRL, → CLSS
7	CLSS (véhicule)	Continuous	(600, 100)	← SAT, → outputs
8	RAND_m bruit y	Sources	(650, 150)	→ SUM y_mes
9	RAND_m bruit ψ	Sources	(650, 200)	→ SUM ψ _mes
10	SUM y_mes	Math	(720, 130)	← y, bruit y, → CTRL
11	SUM ψ _mes	Math	(720, 180)	← ψ , bruit ψ , → CTRL
12	RAND_m bruit r	Sources	(650, 240)	→ SUM r_mes
13	SUM r_mes	Math	(720, 220)	← r, bruit r, → CTRL
14	SCOPE y	Sinks	(800, 50)	← y
15	SCOPE δ	Sinks	(800, 200)	← δ
16	TOWS sim_data	Sinks	(800, 300)	← tous états
17	CLOCK	Sources	(50, 250)	→ scopes (event)
18-34	(blocs internes au superblock)	—	—	—

Annexe B — Code de génération programmatique complet

Cf. fichier `scilab/generate_xcos_diagram.sce` (livré avec ce document).

Annexe C — Visualisation alternative : affichage console

Pour les tests automatisés (Sprint 4), on peut **désactiver** l'interface graphique Xcos et utiliser

`scicos_simulate` directement :

```
// Désactiver l'interface graphique
xset('pixmap', 1);

// Simuler
info = scicos_simulate(scs_m, 'nw'); // nw = no-window
// ou
[state, t] = scicos_simulate(scs_m, 'nw');

// Récupérer les données
load('sim_data.dat'); // depuis TOWS
```

Annexe D — Lien avec la génération de code C (Sprint 4)

En Sprint 4, on utilisera l'outil **Xcos C code generator** pour produire le code C du contrôleur (sans le modèle véhicule, qui sera remplacé par des mesures réelles). Le diagramme Xcos actuel sert de **spécification** pour cette génération.

Pour l'ECU cible (Arduino Mega / Raspberry Pi 4), les paramètres considérés sont :

ECU	Vitesse CPU	RAM	Adapté pour
Arduino Mega	16 MHz	8 KB	✓ Calcul $K \cdot \hat{x}$ (matriciel 4×4)
Arduino Due	84 MHz	96 KB	✓ + Observateur
Raspberry Pi 4	1,5 GHz	4 GB	✓ + Kalman + Logging

Décision Sprint 4 (à arbitrer). Arduino Mega suffit pour le retour d'état. Raspberry Pi 4 préférable pour l'observateur + logging.

Annexe E — Glossaire Xcos

Terme	Définition
<code>scs_m</code>	Structure Scilab représentant un diagramme Xcos
<code>scs_m.props</code>	Propriétés globales (durée simul, solver, etc.)
<code>CLSS</code>	Continuous Linear State Space : système d'état continu
<code>LTI_SYS</code>	Système linéaire (transfer function)
<code>scifunc_block_m</code>	Bloc fonction Scilab générique (pour logique custom)
<code>BIGSOM_f</code>	Bloc SOM_F utilisé pour le superblock
<code>TEXT_f</code>	Bloc texte (annotation)
<code>SUPER_f</code>	Superblock (encapsulation de sous-diagramme)
<code>scicos_diagram</code>	Constructeur d'un nouveau diagramme
<code>scicos_simulate</code>	Lancement de la simulation (headless possible)

Annexe F — Checklist de validation du diagramme

Item	OK	Commentaire
Contexte chargé sans erreur	☐	
Tous les blocs sont créés	☐	
Connexions correctement établies	☐	
Pas de boucle algébrique (CLSS + retour)	☐	vérifié par Scilab
Solveur choisi (Isodar)	☐	
Durée de simulation cohérente	☐	
Scopes ajoutés pour visualisation	☐	
TOWS exporte au moins une variable	☐	
Diagramme s'ouvre dans GUI	☐	
Run s'exécute sans erreur	☐	
Courbes observées correspondent à la théorie	☐	
Sauvegarde .zcos fonctionne	☐	

Validation & workflow Jira

Action	Statut	Date	Owner
Architecture Xcos définie	✅ Done	2026-06-09	Dev 2
Construction programmatique .zcos	✅ Done	2026-06-09	Dev 2
Validation scénario rectiligne	✅ Done	2026-06-09	Dev 2
Validation scénario virage	✅ Done	2026-06-09	Dev 2
Revue croisée par Dev 1	🕒 Sprint Daily	—	Dev 1
Validation Product Owner	🕒 Sprint Review 2	2026-06-27	PO
Génération code C (Sprint 4)	🕒 À démarrer	2026-06-30	Dev 1 + Dev 2

Definition of Done (DoD) — US-301 Diagramme Xcos complet

- ☒ Diagramme Xcos s'ouvre et s'exécute sans erreur
- ☒ Modèle véhicule, observateur, contrôleur, capteurs, saturation présents
- ☒ Scénario rectiligne (REQ-LKA-002) validé
- ☒ Scénario virage (REQ-LKA-006) validé
- ☒ Construction reproductible (programmatische ou documentée)
- ☒ Lien avec la safety ISO 26262 documenté
- ☒ Mode dégradé envisagé (Sprint 3)
- ☐ Validation croisée Dev 1 (en cours)
- ☐ Validation Product Owner (Sprint Review 2)

Fin du document — LKA-ADAS-2026 / Sprint 2 / US-301 : Diagramme Xcos Complet / v1.0

— end of document —